Software Design Document

SMART WASTE MANAGEMENT WEB APPLICATION



Submitted by

ARSHIA BADAR

F23BINFT1M01058

Submitted to

SIR WAHAJ ALI

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Table of Contents

4. Application Architecture 2
Architectural Overview 2
4.1 High-Level Architecture Flow 2
4.2 Overall Architecture Overview 2
4.3 APIs and Libraries 3
5. Data Model Schema 3
5.1 Conceptual Database Design 3
5.2 Data Entities 4
5.3 Entity Relationships 5
6. Implementation 6
6.1 Code Snippet 6
6.2 User Interface 13

4. Application Architecture

Architectural Overview

The Smart Waste Management Web Application uses a Model-View-Controller (MVC)-based architecture, which divides the system into distinct components so each part has a clearly defined responsibility. This separation makes the system easier to maintain, test, and extend. The system operates as follows: the user interacts with the browser, the Flask backend processes the request, the data layer reads or writes JSON files, and the response is rendered and returned to the user.

4.1 High-Level Architecture Flow

User (Browser) → Frontend (HTML/CSS/JS Templates) → Flask Routes (Controllers)

Flask Routes → JSON Data Files (bins.json, routes.json, drivers.json, complaints.json, users.json)

Flask Routes → Scheduling Logic / Route Optimization → Response Generation

Response Generation → Frontend Display → User (Browser)

4.2 Overall Architecture Overview

The system is divided into three main layers, each responsible for a specific set of tasks:

1. Presentation Layer (View)

The Presentation Layer represents the user interface and is implemented using HTML5, CSS3, and JavaScript rendered through Flask's render_template_string function. This layer includes role-specific pages for Admin, Driver, and Citizen users. The interface uses a consistent dark-themed design with a fixed sidebar for navigation, card-based content areas, data tables, forms, and color-coded status badges. The layout is responsive and adapts to different screen sizes through CSS flexbox and grid. All styling is

embedded within the application using a shared STYLE constant to ensure visual consistency across all pages.

2. Application Logic Layer (Controller)

The Application Logic Layer is implemented using Python and the Flask web framework. Flask routes handle incoming HTTP GET and POST requests and coordinate all business logic. This layer manages user authentication and session control, input validation, bin status calculation, schedule generation, route optimization sorting, complaint processing, and data persistence. Each system feature is implemented as a dedicated Flask route function, ensuring clean separation of concerns and easy maintainability.

3. Data Layer (Model)

The Data Layer consists of five JSON files stored in a local data/ directory. These files serve as the persistent data store for the prototype. The load() and save() helper functions provide a consistent interface for reading and writing all data files. The data layer stores user accounts, driver credentials, waste bin records, collection routes and schedules, and citizen complaints. This file-based approach is appropriate for the prototype stage and can be replaced with a relational database such as MySQL in future iterations without changing the controller or presentation layers.

4.3 APIs and Libraries

The system uses the following internal and external components:

Internal Logic

- Bin status calculation: bin_fill_to_status() computes FULL, MEDIUM, or NORMAL based on fill threshold rules
- Route optimization: routes are sorted by pickup bin fill level in descending order on the Routes page
- Auto scheduling: full bins are matched to drivers using modular round-robin index selection
- Session control: admin_required() and driver_required() guard functions protect all restricted routes

External Libraries

- Flask: web framework for HTTP routing, request handling, session management, and template rendering
- Werkzeug: generate_password_hash and check_password_hash for secure password storage and verification
- Python os and json: standard library modules for file system operations and JSON serialization

5. Data Model Schema

5.1 Conceptual Database Design

The system uses a JSON file-based data model appropriate for the prototype stage. Each JSON file represents a logical data entity and stores records as arrays of objects. The data model supports the core

relationships required by the application: bins are monitored and linked to routes, routes are assigned to drivers, and complaints are submitted by registered citizens. All data is accessed through the unified load() and save() helper functions.

5.2 Data Entities

1. Bin:

Stores information about waste bins and their current status in the system.

- id (Automated BinID, e.g., BIN-101)
- location (Physical location of the bin)
- fill (Fill level percentage from 0–100)
- status (Calculated status: FULL, MEDIUM, NORMAL)

2. Route:

Contains details of waste collection tasks and assigned routes for drivers.

- driver (Name of assigned driver)
- route (Description of collection path)
- pickup_from (Bin ID selected for collection)
- bin_location (Location of the bin)
- destination (Waste disposal station)
- status (Task status: Assigned, In Progress, Completed)

3. Driver:

Stores login and identification details of drivers in the system.

- name (Full name of driver)
- username (Login username)
- password (Encrypted password using Werkzeug hashing)

4. Complaint:

Stores complaints submitted by citizens regarding waste management issues.

- name (Citizen name)
- category (Type of complaint such as overflowing bin, missed collection, damaged bin, etc.)
- location (Location related to complaint)
- description (Detailed issue explanation)
- time (Submission timestamp)
- status (Pending, In Progress, Resolved)

5. User (Citizen):

Stores registration and login information of citizens using the system.

- name (Full name of user)

• email (Login email, stored in lowercase)
• password (Encrypted password using Werkzeug hashing)

5.3 Entity Relationships

The entity relationships in the Smart Waste Management Web Application are structured as follows: each Bin record can be associated with one or more Route records when collection tasks are created. Each Route record is linked to exactly one Driver through the driver name field. Each Citizen (User) can submit multiple Complaints, and each Complaint belongs to one Citizen. Routes reference Bins through the pickup_from field, allowing the system to retrieve fill level information for optimization. These relationships ensure structured data flow across the scheduling, routing, and complaint management features.

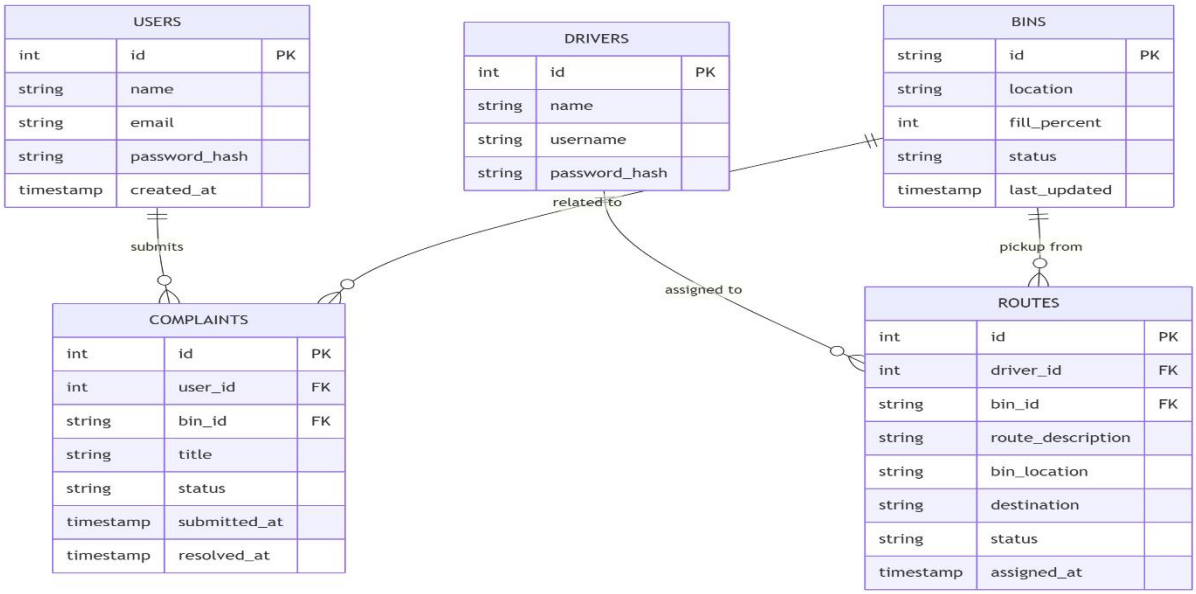


Figure 5.3 ERD Diagram

6. Implementation

The Code Modules of different activities are given below:

6.1 Code Snippet

```python
from flask import Flask, render_template_string, request, redirect, session
import json, os
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.secret_key = "smart_waste_secret_2024"

DATA_FOLDER = "data"
FILES = {
    "drivers":    "drivers.json",
    "complaints": "complaints.json",
    "routes":     "routes.json",
    "bins":       "bins.json",
    "users":      "users.json"
}

# ─────────────────────────────────────────────
# INIT DATA
# ─────────────────────────────────────────────
def initialize():
    os.makedirs(DATA_FOLDER, exist_ok=True)
    defaults = {
        "drivers": [
            {"name": "Ali",   "username": "ali",   "password":
generate_password_hash("1234")},
            {"name": "Ahmed", "username": "ahmed", "password":
generate_password_hash("1234")}
        ],
        "complaints": [],
        "routes": [
            {
                "route": "Sector A → Sector B",
                "driver": "Ali",
                "status": "Assigned",
                "pickup_from": "BIN-101",
                "bin_location": "City Center",
                "destination": "Waste Station A"
            }
        ],
        "bins": [
            {"id": "BIN-101", "location": "City Center", "fill": 90, "status":
"FULL"},
            {"id": "BIN-102", "location": "Park Area",   "fill": 45, "status":
"NORMAL"},
            {"id": "BIN-103", "location": "Main Road",   "fill": 70, "status":
"MEDIUM"},
            {"id": "BIN-104", "location": "East Block",  "fill": 20, "status":
"NORMAL"},
            {"id": "BIN-105", "location": "Bus Stand",   "fill": 85, "status":
"FULL"}
        ],
        "users": []
    }
    for k, f in FILES.items():
        path = os.path.join(DATA_FOLDER, f)
        if not os.path.exists(path):
            with open(path, "w") as file:
                json.dump(defaults[k], file, indent=4)

initialize()

def load(name):
    with open(os.path.join(DATA_FOLDER, FILES[name])) as f:
        return json.load(f)

def save(name, data):
    with open(os.path.join(DATA_FOLDER, FILES[name]), "w") as f:
        json.dump(data, f, indent=4)

# ─────────────────────────────────────────────
# HELPERS
# ─────────────────────────────────────────────
def bin_fill_to_status(fill_int):
    if fill_int >= 80:   return "FULL"
    elif fill_int >= 50: return "MEDIUM"
    else:                return "NORMAL"

def status_color(status):
    return {"FULL": "#ef4444", "MEDIUM": "#f59e0b", "NORMAL":
"#22c55e"}.get(status, "#64748b")

def admin_required():
    if not session.get("admin"):
        return redirect("/admin/login")
    return None

def driver_required():
    if not session.get("driver"):
        return redirect("/driver/login")
```

```
    return None

# ─────────────────────────────────────────
# SHARED STYLE
# ─────────────────────────────────────────
STYLE = """
<meta name='viewport' content='width=device-width, initial-scale=1'>
<style>
*{box-sizing:border-box;margin:0;padding:0;}
body{font-family:Arial,sans-serif;background:#0f172a;color:#e2e8f0;min-
height:100vh;}
a{color:#22c55e;text-decoration:none;}
h1{font-size:1.8rem;margin-bottom:16px;}
h2{font-size:1.3rem;margin-bottom:12px;}
h3{font-size:1.1rem;margin-bottom:8px;}

/* Layout */
.sidebar{width:220px;height:100vh;position:fixed;top:0;left:0;background:#111827;
padding:20px;overflow-y:auto;z-index:100;}
.sidebar h2{color:#22c55e;font-size:1.1rem;margin-bottom:20px;}
.sidebar a{display:block;color:#cbd5e1;padding:10px 12px;border-
radius:8px;margin-bottom:4px;transition:background .2s;}
.sidebar a:hover{background:#1e293b;color:white;}
.main{margin-left:230px;padding:28px;}

/* Cards */
.card{background:#1e293b;padding:18px;margin:10px 0;border-radius:12px;border:1px
solid #334155;}
.cards-row{display:flex;flex-wrap:wrap;gap:16px;margin-bottom:20px;}
.stat-card{background:#1e293b;padding:20px;border-radius:12px;border:1px solid
#334155;flex:1;min-width:140px;text-align:center;}
.stat-card .num{font-size:2rem;font-weight:bold;color:#22c55e;}
.stat-card .lbl{color:#94a3b8;font-size:.85rem;margin-top:4px;}

/* Forms */
.form-box{background:#1e293b;padding:24px;border-radius:12px;border:1px solid
#334155;max-width:480px;}
input,select,textarea{width:100%;padding:10px 12px;margin-top:8px;border-
radius:8px;border:1px solid #334155;background:#0f172a;color:white;font-
size:14px;}
input:focus,select:focus{outline:none;border-color:#22c55e;}
label{font-size:13px;color:#94a3b8;margin-top:12px;display:block;}

/* Buttons */
.btn{background:#22c55e;padding:10px 20px;border:none;color:white;border-
radius:8px;cursor:pointer;font-size:14px;font-weight:600;display:inline-
block;transition:background .2s;}
.btn:hover{background:#16a34a;}
.btn-red{background:#ef4444;} .btn-red:hover{background:#dc2626;}
.btn-blue{background:#3b82f6;} .btn-blue:hover{background:#2563eb;}
.btn-sm{padding:6px 12px;font-size:12px;}

/* Alerts */
.alert{padding:12px 16px;border-radius:8px;margin:10px 0;font-size:14px;}
.alert-green{background:#14532d;border:1px solid #22c55e;color:#86efac;}
.alert-red{background:#450a0a;border:1px solid #ef4444;color:#fca5a5;}
.alert-yellow{background:#431407;border:1px solid #f59e0b;color:#fde68a;}

/* Badges */
.badge{display:inline-block;padding:3px 10px;border-radius:20px;font-
size:12px;font-weight:600;}
.badge-full{background:#450a0a;color:#fca5a5;}
.badge-medium{background:#431407;color:#fde68a;}
.badge-normal{background:#14532d;color:#86efac;}
.badge-pending{background:#1e1b4b;color:#a5b4fc;}
.badge-assigned{background:#1e3a5f;color:#93c5fd;}
.badge-completed{background:#14532d;color:#86efac;}

/* Table */
table{width:100%;border-collapse:collapse;font-size:14px;}
th{background:#334155;padding:10px 14px;text-align:left;color:#94a3b8;font-
weight:600;}
td{padding:10px 14px;border-bottom:1px solid #1e293b;}
tr:hover td{background:#1e293b;}

/* Hero */
.hero{min-height:100vh;display:flex;flex-direction:column;justify-
content:center;align-items:center;text-align:center;padding:20px;}
.hero h1{font-size:3rem;margin-bottom:12px;}
.hero-cards{display:flex;gap:24px;flex-wrap:wrap;justify-content:center;margin-
top:30px;}
.hero-card{background:#1e293b;padding:30px 40px;border-radius:16px;border:1px
solid #334155;min-width:180px;text-align:center;transition:border-color .2s;}
.hero-card:hover{border-color:#22c55e;}
.hero-card h2{font-size:2rem;margin-bottom:8px;}

/* Progress bar */
.bar-bg{background:#334155;border-radius:20px;height:10px;margin-top:6px;}
```

```
</style>
"""

# ─────────────────────────────────────────────
# SIDEBARS
# ─────────────────────────────────────────────
def admin_sidebar():
    return """
    <div class='sidebar'>
        <h2>⚙️ Admin Panel</h2>
        <a href='/admin/dashboard'>📊 Dashboard</a>
        <a href='/admin/bins'>🗑️ Bin Monitoring</a>
        <a href='/admin/sensor'>📡 IoT Sensor Input</a>
        <a href='/admin/auto_schedule'>🤖 Auto Schedule</a>
        <a href='/admin/assign'>📋 Manual Assign</a>
        <a href='/admin/routes'>🗺️ Routes</a>
        <a href='/admin/add_driver'>🚛 Add Driver</a>
        <a href='/admin/complaints'>📣 Complaints</a>
        <a href='/logout'>🚪 Logout</a>
    </div>
    """

def driver_sidebar(name=""):
    return f"""
    <div class='sidebar'>
        <h2>🚛 Driver Panel</h2>
        <p style='color:#94a3b8;font-size:13px;margin-bottom:16px;'>Hello,
{name}</p>
        <a href='/driver/dashboard'>📋 My Tasks</a>
        <a href='/logout'>🚪 Logout</a>
    </div>
    """

# ─────────────────────────────────────────────
# HOME
# ─────────────────────────────────────────────
@app.route("/")
def home():
    return render_template_string(f"""
    {STYLE}
    <div class='hero'>
        <h1>♻️ Smart Waste Management</h1>
        <p style='color:#94a3b8;font-size:1.1rem;'>Select your role to
continue</p>
        <div class='hero-cards'>
```

```
            <a href="/admin/login" class="hero-card">
                <h2>🔐</h2>
                <p style="font-weight:600;">Admin</p>
                <p style="color:#94a3b8;font-size:13px;margin-top:6px;">Manage
system</p>
            </a>
            <a href="/driver/login" class="hero-card">
                <h2>🚛</h2>
                <p style="font-weight:600;">Driver</p>
                <p style="color:#94a3b8;font-size:13px;margin-top:6px;">View
routes</p>
            </a>
            <a href="/citizen" class="hero-card">
                <h2>👤</h2>
                <p style="font-weight:600;">Citizen</p>
                <p style="color:#94a3b8;font-size:13px;margin-top:6px;">Submit
complaints</p>
            </a>
        </div>
    </div>
    """)

# ─────────────────────────────────────────────
# CITIZEN REGISTRATION & LOGIN
# ─────────────────────────────────────────────
@app.route("/register", methods=["GET", "POST"])
def register():
    error = ""
    success = ""
    if request.method == "POST":
        name     = request.form.get("name", "").strip()
        email    = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")
        confirm  = request.form.get("confirm", "")

        if not name or not email or not password:
            error = "All fields are required."
        elif "@" not in email:
            error = "Enter a valid email address."
        elif len(password) < 4:
            error = "Password must be at least 4 characters."
        elif password != confirm:
            error = "Passwords do not match."
        else:
            users = load("users")
```

```
            if any(u["email"] == email for u in users):
                error = "An account with this email already exists."
            else:
                users.append({
                    "name": name,
                    "email": email,
                    "password": generate_password_hash(password)
                })
                save("users", users)
                success = "Registration successful! You can now log in."

    return render_template_string(f"""
    {STYLE}
    <div style="min-height:100vh;display:flex;align-items:center;justify-
content:center;padding:20px;">
        <div class="form-box" style="width:100%;">
            <h1 style="text-align:center;margin-bottom:4px;">♻️ Register</h1>
            <p style="text-align:center;color:#94a3b8;margin-bottom:20px;font-
size:14px;">Create a citizen account</p>
            {"<div class='alert alert-red'>"+error+"</div>" if error else ""}
            {"<div class='alert alert-green'>"+success+"</div>" if success else
""}
            <form method="post">
                <label>Full Name</label>
                <input name="name" placeholder="Your full name" required>
                <label>Email Address</label>
                <input name="email" type="email" placeholder="you@email.com"
required>
                <label>Password</label>
                <input name="password" type="password" placeholder="Min 4
characters" required>
                <label>Confirm Password</label>
                <input name="confirm" type="password" placeholder="Repeat
password" required>
                <button class="btn" style="width:100%;margin-top:18px;">Create
Account</button>
            </form>
            <p style="text-align:center;margin-top:16px;font-
size:13px;color:#94a3b8;">Already have an account? <a
href="/citizen/login">Login</a></p>
            <p style="text-align:center;margin-top:8px;"><a href="/">← Back to
Home</a></p>
        </div>
    </div>
    """)
```

```
@app.route("/citizen/login", methods=["GET", "POST"])
def citizen_login():
    error = ""
    if request.method == "POST":
        email    = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")
        users = load("users")
        user = next((u for u in users if u["email"] == email), None)
        if user and check_password_hash(user["password"], password):
            session["citizen"] = user["email"]
            session["citizen_name"] = user["name"]
            return redirect("/citizen")
        error = "Invalid email or password."

    return render_template_string(f"""
    {STYLE}
    <div style='min-height:100vh;display:flex;align-items:center;justify-
content:center;padding:20px;'>
        <div class='form-box' style='width:100%;'>
            <h1 style='text-align:center;margin-bottom:4px;'>👤 Citizen
Login</h1>
            <p style='text-align:center;color:#94a3b8;margin-bottom:20px;font-
size:14px;'>Sign in to your account</p>
            {"<div class='alert alert-red'>"+error+"</div>" if error else ""}
            <form method='post'>
                <label>Email Address</label>
                <input name='email' type='email' placeholder='you@email.com'
required>
                <label>Password</label>
                <input name='password' type='password' placeholder='Your
password' required>
                <button class='btn' style='width:100%;margin-
top:18px;'>Login</button>
            </form>
            <p style='text-align:center;margin-top:16px;font-
size:13px;color:#94a3b8;'>No account? <a href='/register'>Register here</a></p>
            <p style='text-align:center;margin-top:8px;'><a href='/'>← Back to
Home</a></p>
        </div>
    </div>
    """)

# ─────────────────────────────────────────
# ADMIN LOGIN
```

```
# ─────────────────────────────────────────
@app.route("/admin/login", methods=["GET", "POST"])
def admin_login():
    error = ""
    if request.method == "POST":
        if request.form.get("username") == "admin" and
request.form.get("password") == "admin123":
            session["admin"] = True
            return redirect("/admin/dashboard")
        error = "Invalid credentials."
    return render_template_string(f"""
    {STYLE}
    <div style='min-height:100vh;display:flex;align-items:center;justify-
content:center;padding:20px;'>
        <div class='form-box' style='width:100%;'>
            <h1 style='text-align:center;margin-bottom:4px;'>🗑️ Admin Login</h1>
            {"<div class='alert alert-red'>"+error+"</div>" if error else ""}
            <form method='post'>
                <label>Username</label>
                <input name='username' placeholder='admin'>
                <label>Password</label>
                <input name='password' type='password' placeholder='••••••••'>
                <button class='btn' style='width:100%;margin-
top:18px;'>Login</button>
            </form>
            <p style='text-align:center;margin-top:12px;'><a href='/'>← Back to
Home</a></p>
        </div>
    </div>
    """)

def get_bin_rows(bins):
    rows = ""
    for b in bins:
        rows += f"<tr><td>{b['id']}</td><td>{b['location']}</td><td><div
class='bar-bg'><div class='bar-fill'
style='width:{b['fill']}%;background:{status_color(b['status'])};'></div></div><s
mall>{b['fill']}%</small></td><td><span class='badge badge-
{b['status'].lower()}'>{b['status']}</span></td></tr>"
    return rows

# ─────────────────────────────────────────
# ADMIN DASHBOARD
# ─────────────────────────────────────────
@app.route("/admin/dashboard")
```

```
def dashboard():
    if not session.get("admin"):
        return redirect("/admin/login")
    bins       = load("bins")
    drivers    = load("drivers")
    complaints = load("complaints")
    routes     = load("routes")
    users      = load("users")

    full_bins       = sum(1 for b in bins if b["status"] == "FULL")
    pending_comps   = sum(1 for c in complaints if c.get("status") == "Pending")
    active_routes   = sum(1 for r in routes if r.get("status") == "Assigned")
    done_routes     = sum(1 for r in routes if r.get("status") == "Completed")

    return render_template_string(f"""
    {STYLE}
    {admin_sidebar()}
    <div class='main'>
        <h1>📊 Admin Dashboard</h1>
        <div class='cards-row'>
            <div class='stat-card'><div class='num'>{len(bins)}</div><div
class='lbl'>Total Bins</div></div>
            <div class='stat-card'><div class='num'
style='color:#ef4444;'>{full_bins}</div><div class='lbl'>Full Bins</div></div>
            <div class='stat-card'><div class='num'>{len(drivers)}</div><div
class='lbl'>Drivers</div></div>
            <div class='stat-card'><div class='num'
style='color:#f59e0b;'>{pending_comps}</div><div class='lbl'>Pending
Complaints</div></div>
            <div class='stat-card'><div class='num'
style='color:#3b82f6;'>{active_routes}</div><div class='lbl'>Active
Routes</div></div>
            <div class='stat-card'><div class='num'>{done_routes}</div><div
class='lbl'>Completed Routes</div></div>
            <div class='stat-card'><div class='num'>{len(users)}</div><div
class='lbl'>Citizens</div></div>
        </div>
        <div class='card'>
            <h2>🗑️ Bin Status Overview</h2>
            <table>
                <tr><th>Bin ID</th><th>Location</th><th>Fill
Level</th><th>Status</th></tr>
                {get_bin_rows(bins)}
            </table>
        </div>
```

6.2 User Interface

1. Design Philosophy

The system follows a clean, modern waste management dashboard design. A dark color scheme with a deep navy/slate background (#0F172A, #1E293B) provides a professional appearance suited for an operational management system. Green (#22C55E) is used as the primary accent color for actionable elements, success states, and navigation highlights, reinforcing the environmental theme. A card-based layout organizes information into distinct sections for better readability. Navigation uses a fixed sidebar appropriate for data-heavy administrative interfaces.

2. Screen Constraints

The application is developed as a web-based Flask system running on standard desktop browsers including Chrome and Firefox. The interface is mobile-responsive through CSS flexbox and grid, with a fixed sidebar on desktop and a stacked layout on smaller screens. Viewport meta tags ensure correct scaling on mobile devices.

3. Screens / Pages

Home Page (Role Selection)

The landing page displays a hero layout with three role selection cards: Admin, Driver, and Citizen. Each card links to the appropriate login page and displays a role icon, role name, and brief description.

Registration Page

The Citizen Registration Page allows new users to create an account. It contains fields for Full Name, Email Address, Password, and Confirm Password. The system validates all fields, checks email format, enforces minimum password length, confirms password match, and rejects duplicate email addresses with clear error messages.

Login Pages (Admin / Driver / Citizen)

Each role has a dedicated login page. The Admin login accepts a fixed username and password. The Driver login checks credentials against drivers.json. The Citizen login verifies email and hashed password from users.json. All login pages show error messages for invalid credentials and provide a back link to the home page.

Admin Dashboard

The Admin Dashboard displays summary statistics in a row of stat cards: total bins, full bins, medium bins, normal bins, total drivers, routes, and complaints. A current bin status table shows all bins at a glance. The sidebar provides navigation to all admin features.

Bin Monitoring Page

The Bin Monitoring Page displays all waste bins in a responsive card grid. Each card shows the bin ID, location, fill percentage as a colored progress bar, and a status badge (FULL in red, MEDIUM in amber, NORMAL in green).

IoT Sensor Input Page

The IoT Sensor Input Page provides a form where the admin can enter a bin ID and a new fill level value to simulate sensor updates. The system validates the bin ID, updates the fill level, recalculates the status, and confirms the update.

Auto Schedule Generation Page

This page lists all currently FULL bins in a table and provides a Generate Collection Schedule button. When clicked, the system creates collection tasks for all full bins and assigns drivers automatically. The page shows a success message and lists all generated assignments.

Routes Page

The Routes Page displays all collection tasks in a table sorted by bin fill level (highest first). Each row shows driver name, route description, pickup bin ID, bin location, destination, fill percentage, and status badge.

Driver Dashboard

The Driver Dashboard shows only the routes assigned to the currently logged-in driver. Each task card displays the route description, pickup bin, bin location, destination, and status badge.

Citizen Portal and Complaint Form

The Citizen Portal shows current bin statuses and provides a complaint submission form. The form includes fields for name, complaint category (dropdown), location, and description. Submitted complaints are saved and shown in a confirmation message.

4. User Interaction Flow

The complete interaction flow proceeds as follows:

1. User opens the application and sees the home page with three role cards
2. Admin logs in, reviews bin statuses on the dashboard, updates fill levels via IoT Sensor Input, generates collection schedules, reviews optimized routes, manages drivers, and resolves citizen complaints
3. Driver logs in, views their assigned collection tasks on the Driver Dashboard, and follows route instructions
4. Citizen registers or logs in, views current bin statuses, and submits complaints with category and location details
5. All users log out securely when finished